

The idea of EchoCare originated from a conversation with an acquaintance working at a healthcare center for people with dementia. She described how music therapy was used in practice, which motivated me to explore academic research on the topic. The more I read, the more ambitious I became, and I initially designed a detailed and comprehensive project plan. After submitting the project proposal (arbeidskrav), I was determined to implement the system thoroughly. However, due to personal mental health challenges, I was unable to work on the project for a significant period. Regaining the energy to return and complete the system therefore became a meaningful achievement in itself.

Throughout development, I made several architectural and system design decisions, continuously balancing ambition with practical constraints such as limited time. To reduce implementation complexity, I deliberately simplified parts of the data model. For example, I used `List<String>` instead of enums for fields such as `dementiaStage` (mild, moderate, severe) and `careNeed` (stress_relief, activity_support, calming_agitation, easing_depression, reducing_anxiety). While enums would provide stronger type safety and clearer domain constraints, the simpler structure reduced boilerplate code and allowed faster iteration within the available timeframe.

I also revised the Feedback entity by removing `songId`. Initially, I considered collecting feedback on individual songs. However, I realized that caregivers are primarily concerned with the overall therapeutic effect of a playlist rather than evaluating each song separately. This decision improved alignment with the core purpose of the application and simplified both the API design and data model, although it reduced the granularity of collected feedback data.

Regarding persistence, I initially configured Docker with automatic database seeding using `hibernate.ddl-auto=create-drop`. Because I chose UUIDs as primary keys, repeated auto-seeding caused duplicate key errors and prevented reliable service startup. To address this, I migrated to Flyway-based schema management, ensuring controlled migrations and stable seed data. This transition improved reliability and brought the project closer to production-oriented practices.

While experimenting with Spring Cloud Config Server as a centralized configuration source, I encountered configuration precedence issues. Although the service successfully connected to the Config Server with the “docker” profile active, both the local `application-docker.yml` file and the `SERVER_PORT` environment variable in `docker-compose.yml` overrode the remote configuration. This experience strengthened my understanding of Spring Boot’s configuration hierarchy. By removing the conflicting local configuration and environment variables, I ensured that the Config Server functioned as the single source of truth.

Finally, I actively used AI as a development partner during this project. While AI assistance increased productivity—particularly for documentation, boilerplate code, and structural suggestions—it also introduced additional complexity. Some generated solutions required significant debugging and refactoring before they worked correctly within my architecture. This experience taught me that AI is most effective when used critically and selectively. Rather than replacing understanding, it required me to deepen my own technical reasoning

in order to evaluate, adapt, and integrate the suggestions responsibly. I disclose this use transparently and with a clear awareness of its limitations and benefits.